

upon the usage. You can define an accessor and mutator by using a `getFooAttribute()` and `setFooAttribute()` methods respectively, where 'Foo' is the name of the column. The sample code for defining an accessor will look like as shown below:

```
public function getFirstNameAttribute($value)
{
    return ucfirst($value);
}
```

As you can see, we have replaced foo with the relevant column name. If you are defining a mutator, you should use something like as shown below.

```
public function setFirstNameAttribute($value)
{
    $this->attributes['first_name'] = strtolower($value);
}
```

With Eloquent, you can customize the dates that are mutated automatically by overriding the `$dates` property. You will have to use `created_at`, `updated_at` and `deleted_at` columns. To customize the format of the timestamp, you need to set the `$dateFormat` property. It is possible to convert attributes to commonly used data types such as integer, float, double, string, array, date and timestamp using `#casts` property. You can also cast Array and JSON by using the options attribute.



Working with Accessors and Mutators

API Resources

Laravel provides several resource classes using which you can transform models and related collections into JSON. You can generate a resource class using `make:resource` Artisan command as shown below:

```
php artisan make:resource UserResource1
```